

Project Design Phase Proposed Solution Template

Date	18 June 2026
Name	Dhanish Ladwani
Project Name	ShopEZ – E-Commerce Application
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1	Problem Statement (Problem to be solved)	Everyday online shoppers struggle to find a shopping platform that is both trustworthy and easy to use — most either belong to large generic marketplaces with high seller fees, or to informal, unorganized channels (e.g., WhatsApp/Instagram DM orders) that offer no order tracking, no secure checkout, and no transparency on delivery. At the same time, small and medium online sellers/admins lack an affordable, easy-to-manage platform to list products, process orders and view sales performance, forcing them to rely on manual spreadsheets or chat-based order taking, which is error-prone and impossible to scale.
2	Idea / Solution description	ShopEZ is a full-stack MERN (MongoDB, Express, React, Node.js) e-commerce web application that gives customers a fast, secure, and transparent shopping experience, and gives sellers/admins a single dashboard to run their store. - Customer side: secure JWT-based login/signup, searchable & filterable product catalog (by category, gender, price, rating), dynamic shopping cart, wishlist, and a transparent checkout-to-delivery order tracking flow. - Admin side: a real-time analytics dashboard (revenue, order distribution, top-selling products, user growth), a product builder with live preview, and order/user management with status controls. - Security: bcrypt password hashing, JWT authentication, Helmet-secured HTTP headers, and rate limiting on auth and API routes to prevent abuse.
3	Novelty / Uniqueness	- A bespoke, hand-built vanilla-CSS design system rather than a generic UI-library look, giving the storefront a unique, lightweight, and fast-loading interface. - Combines a full customer storefront and a complete admin operations dashboard in one unified codebase, instead of stitching together a marketplace listing tool and a separate analytics add-on.

		• Production-oriented security (rate limiting, Helmet, JWT, hashed passwords) built in from the start, which most student/demo e-commerce projects skip. • One-command database seeding (sample products, users, and historical orders) lets the whole platform — including analytics charts — be demonstrated instantly without manual data entry.
4	Social Impact / Customer Satisfaction	• Gives small sellers who cannot afford expensive SaaS platforms or marketplace commissions an accessible way to run a professional online store. • Builds customer trust and satisfaction through transparent pricing, secure authentication, and visible order status/delivery timelines — reducing the anxiety associated with unfamiliar or informal online sellers. • Wishlist and review/rating visibility help customers make more informed purchase decisions, improving overall shopping confidence.
5	Business Model (Revenue Model)	• Commission-based: a small percentage fee on each order processed through the platform, charged to sellers/admins using the storefront. • Subscription tiers for admins: a free tier with basic catalog/order management, and paid tiers unlocking advanced analytics, more product listings, or premium homepage banner placement. • Featured listings: sellers can pay to have products highlighted on the home page or appear higher in search/category results. • Optional value-added services such as payment gateway integration fees or premium customer support for admin users.
6	Scalability of the Solution	• Decoupled MERN architecture (React/Vite client + Express REST API + MongoDB Atlas) allows the frontend and backend to be scaled independently as traffic grows. • MongoDB Atlas cloud database scales horizontally with sharding/replica sets, supporting growth in products, users, and orders without re-architecture. • Stateless JWT authentication and a modular MVC backend (controllers, routes, middleware) make it straightforward to add new features (e.g., payments, notifications) or scale to multiple categories, vendors, or even a multi-vendor marketplace model. • Built-in rate limiting, compression, and security middleware mean the same codebase can be deployed for a small store today and a larger multi-seller platform later with minimal rework.